

order validation

```
package jp.co.activekenshu.delivery.validator;
```

```
import jp.co.activekenshu.delivery.dto.OrderDto;
```

```
/**
 * 配送依頼情報の入力チェックを行うクラスです。
 *
 * <p>
 * このクラスでは、配送依頼画面で入力された値に対して、
 * 必須チェックと文字数チェックを行います。
 * </p>
 *
 * <p>
 * 文字数制限は、delivery_order テーブルのカラム定義に合わせています。
 * </p>
 */
public class OrderValidator {
```

```
    /**
     * 依頼主名、配送先名の最大文字数です。
     *
     * <p>
     * DB の以下のカラムに対応しています。
     * </p>
     *
     * <ul>
     * <li>customer_name VARCHAR2(120)</li>
     * <li>delivery_name VARCHAR2(120)</li>
     * </ul>
     */
    private static final int NAME_MAX_LENGTH = 120;
```

```
    /**
     * 依頼主住所、配送先住所の最大文字数です。
     *
     * <p>
     * DB の以下のカラムに対応しています。
     * </p>
     */
```

```

* <ul>
*   <li>customer_address VARCHAR2(200)</li>
*   <li>delivery_address VARCHAR2(200)</li>
* </ul>
*/
private static final int ADDRESS_MAX_LENGTH = 200;

/**
 * 配送依頼情報をチェックします。
 *
 * <p>
 * エラーがある場合は IllegalArgumentException を投げます。
 * この例外は ConfirmServlet で catch され、order.jsp にエラーメッセージとして表示されます。
 * </p>
 *
 * @param order 配送依頼情報
 * @throws IllegalArgumentException 入力値に問題がある場合
 */
public void validate(OrderDto order) {

    // DTO 自体が null の場合は、画面から入力情報を受け取れていないためエラーにします。
    if (order == null) {
        throw new IllegalArgumentException("配送依頼情報が取得できませんでした。");
    }

    // 依頼主名のチェック
    validateRequired(order.getCustomerName(), "依頼主名");
    validateMaxLength(order.getCustomerName(), NAME_MAX_LENGTH, "依頼主名");

    // 依頼主住所のチェック
    validateRequired(order.getCustomerAddress(), "依頼主住所");
    validateMaxLength(order.getCustomerAddress(), ADDRESS_MAX_LENGTH, "依頼主住所");

    // 配送先名のチェック
    validateRequired(order.getDeliveryName(), "配送先名");
    validateMaxLength(order.getDeliveryName(), NAME_MAX_LENGTH, "配送先名");

    // 配送先住所のチェック
    validateRequired(order.getDeliveryAddress(), "配送先住所");
    validateMaxLength(order.getDeliveryAddress(), ADDRESS_MAX_LENGTH, "配送先住所");
}

```

```
/**
 * 必須入力チェックを行います。
 *
 * @param value チェックする値
 * @param itemName 項目名
 * @throws IllegalArgumentException 未入力の場合
 */
private void validateRequired(String value, String itemName) {

    // null、空文字、半角スペースだけ、全角スペースだけの場合はエラーにします。
    if (isBlank(value)) {
        throw new IllegalArgumentException(itemName + "を入力してください。");
    }
}
```

```
/**
 * 最大文字数チェックを行います。
 *
 * @param value チェックする値
 * @param maxLength 最大文字数
 * @param itemName 項目名
 * @throws IllegalArgumentException 最大文字数を超過している場合
 */
private void validateMaxLength(String value, int maxLength, String itemName) {

    // nullの場合はここではチェックしません。
    // 必須チェックはvalidateRequiredで行います。
    if (value == null) {
        return;
    }

    // 入力された文字数が最大文字数を超過していたらエラーにします。
    if (value.length() > maxLength) {
        throw new IllegalArgumentException(
            itemName + "は" + maxLength + "文字以内で入力してください。"
        );
    }
}
```

```
/**
```

```
* 未入力かどうかを判定します。
*
* @param value 判定する文字列
* @return null、空文字、スペースのみの場合 true
*/
private boolean isBlank(String value) {

    if (value == null) {
        return true;
    }

    // 半角スペースと全角スペースを取り除いて、空かどうかを確認します。
    return value.trim().replace(" ", "").isEmpty();
}
}
```